

Bitácora de Prompts: Proyecto LuciApp

Curso: Sistemas de Información - Ingeniería Comercial

Equipo: Emilio Tabilo, Constanza Labarca, Sarhavi Vega, Héctor Rojas

1. Introducción

Como estudiantes de negocios, desarrollar una aplicación funcional desde cero representaba un desafío técnico fuera de nuestra área tradicional. Por ello, utilizamos la Inteligencia Artificial no solo como un buscador, sino como nuestro equipo de programación (Pair-Programming). Esta bitácora documenta cómo fuimos iterando nuestras instrucciones (Prompts) en las distintas fases del proyecto, aprendiendo de nuestros errores para lograr que la IA construyera la solución exacta que Lucía necesitaba.

2. Fase de Descubrimiento y Requerimientos (NotebookLM)

Objetivo: Entender la problemática financiera de Lucía (Caso 04) antes de pensar en la tecnología.

-  Prompt Inicial (Prueba 1): *"Resume el caso de Lucía y dime qué aplicación le podemos hacer."*
 - *Resultado:* La respuesta fue muy genérica y obvia, no nos sirvió para definir qué botones o pantallas íbamos a necesitar.
 -  Prompt Optimizado (Rol de Negocios): *"Actúa como un consultor financiero. Analiza el 'Caso 04: Freelance en Apuros'. Identifica en qué partes exactas Lucía tiene 'fugas de ingresos' y cuellos de botella por su desorden manual. Con base en ese diagnóstico, hazme una lista detallada de los requerimientos funcionales que debe tener un sistema digital para solucionar esto."*
 - *Resultado:* Excelente. La IA ordenó las ideas y nos dijo exactamente qué módulos armar: un panel financiero, un cronómetro integrado y un portal de autogestión para sus clientes.
-

3. Fase de Prototipado Visual (Stitch)

Objetivo: Pre-visualizar la app para entender cómo iba a navegar la usuaria.

-  Prompt para el Panel de Lucía: *"Diseña el panel principal (Dashboard) para una diseñadora freelance. El estilo debe ser moderno y muy limpio. Arriba necesito tres grandes indicadores: Dinero Facturado este mes, Dinero por Cobrar (en color rojo) y Horas Trabajadas. Abajo, pon una lista visual que muestre alertas de clientes que tienen facturas atrasadas más de 30 días (como el cliente de enero en nuestro caso de estudio)."*

-  Prompt para el Portal del Cliente: *"Ahora diseña cómo vería la app el cliente de Lucía. Quiero una vista donde el cliente vea el progreso de su diseño en porcentaje, un resumen de las horas consumidas y un botón muy visible que diga 'Pagar Ahora'. La idea es que al cliente no le cueste trabajo pagar su deuda."*
-

4. Fase de Desarrollo de Código con IA (Gemini / AI Studio)

Objetivo: Pasar las imágenes a código real en React (Frontend) y darle vida a los botones. Aquí fue donde descubrimos la importancia de pedir las cosas "paso a paso".

- Iteración 1: La estructura (El "Cascarón")
 -  Prompt: *"Toma el rol de un desarrollador web experto. Vamos a crear la aplicación 'LuciApp' en React usando Tailwind CSS. Necesito primero el diseño general: Crea un menú lateral izquierdo con iconos para: Inicio, Clientes, Facturas y Ajustes. En el centro, arma las 3 tarjetas de métricas financieras que te pedí antes."*
- Iteración 2: El problema del Celular (Diseño Mobile-First)
 - *El Problema:* El código anterior funcionaba perfecto en el computador, pero al verlo en el celular de prueba, el menú tapaba toda la pantalla y las gráficas se cortaban.
 -  Prompt de Corrección: *"Tenemos un problema. El código se rompe cuando achicamos la pantalla del navegador a tamaño teléfono. El menú lateral estorba. Ajusta las clases de Tailwind para que el diseño sea 'responsivo'. Cuando sea pantalla pequeña, oculta el menú lateral y conviértelo en una barra de navegación abajo (como en las apps de celular). Haz que las tablas se apilen hacia abajo."*
 - *Resultado:* La IA introdujo clases como sm: y flexbox, arreglando la vista.
- Iteración 3: Hacer que el Cronómetro funcione de verdad
 - *El Problema:* La IA nos había dado el botón del reloj, pero era de "adorno", no contaba el tiempo.
 -  Prompt de Lógica: *"Necesito que el cronómetro funcione de verdad con código de React (useState). Debe tener un botón de Start y otro de Stop. Cuando le doy Start, los segundos tienen que ir avanzando en la pantalla. Además, guarda el tiempo total cuando le dé a Stop en una variable para después cobrar esas horas."*
- Iteración 4: Simulando la pasarela de pago

- *El Problema:* Faltaba demostrar cómo el cliente paga y cómo eso actualiza las finanzas de Lucía.
 -  Prompt de Lógica de Negocio: *"Vamos a hacer el flujo de pago. En el Portal del Cliente, cuando alguien presione el botón 'Pagar Factura', quiero que cambie el texto a 'Procesando...' por 2 segundos. Luego, que cambie a verde con el texto 'Pagado'. En ese mismo instante, dispara una alerta visual pequeña (Toast) en la esquina que diga 'Pago exitoso', para que simule la experiencia real."*
-

5. Nuestras Conclusiones sobre Prompt Engineering

Como estudiantes sin experiencia previa en programación, concluimos que las herramientas de IA no son "magia que hace apps enteras con una frase". Nuestra receta para el éxito fue:

1. Asignarle un Rol (Rol-play): Decirle a la IA "Actúa como un arquitecto de software" hacía que el código que nos entregaba fuera mucho más ordenado.
2. Darle Contexto de Negocio: Cuando la IA sabe *por qué* estamos haciendo el botón (ej. "Para evitar fugas de ingresos de Lucía"), programa componentes mucho más útiles y alineados al caso.
3. Dividir y Vencerás: El peor error que cometimos al inicio fue pedirle "hazme la app de facturación". Aprendimos que hay que pedir primero la pantalla principal, luego la vista del celular, luego el reloj, y por último los colores. La paciencia rinde frutos.